

Introduction à l'algorithmique et la programmation

Dr Ibrahima GAYE - Université Alioune DIOP de Bambey

30 mars 2021

CHAPITRE 2 : OBJETS ET INSTRUCTIONS

Table des matières

1	NOTION D'OBJET	3
1.1	Identificateur	4
1.2	Type	4
1.3	Déclaration des variables	5
1.4	Les types primitifs	5
1.5	Syntaxe générale de l'algorithmique	8
2	INSTRUCTIONS ELEMENTAIRES	9
2.1	Généralités	9
2.2	Affectation	9
2.2.1	Présentation	9
2.2.2	Distinguer l'affectation de l'égalité mathématique	10
2.2.3	Compléments	11
2.3	Entrée et Sortie	12
2.3.1	L'instruction d'écriture (entrées)	13
2.3.2	L'instruction de lecture (sortie)	14
2.3.3	Synthèse	15

1 NOTION D'OBJET

Un algorithme a pour but de traiter une ou plusieurs donnée(s) afin de produire un ou des résultat(s). Les résultats peuvent eux-mêmes être des données pour d'autres algorithmes.

$$\text{Donnée (s)} \implies \text{programme} \implies \text{résultat(s)}$$

Les données et les résultats sont regroupés sous le nom d'objets. Pour utiliser un objet, il est nécessaire de pouvoir circonscrire l'ensemble des valeurs que peut prendre l'objet. Traiter un objet signifie traiter la valeur de cet objet.

Si cette valeur ne peut pas être modifiée, on dit que l'objet est une **constante**, sinon on dit que l'objet est une **variable**.

Les données d'un algorithme doivent être récupérées en mémoire centrale, à partir du clavier ou d'un fichier par exemple, pour pouvoir être traitées par le processeur qui exécute le programme. Ainsi, toutes les données d'un programme sont mémorisées en mémoire centrale, dans des sortes de cases que l'on appelle variables.

Une variable peut être représentée par une case mémoire, qui contient la valeur d'une donnée. C'est un nom qui sert à repérer un placement donné dans la mémoire centrale.

Donc une variable en tant qu'emplacement va être considérée comme un contenant. Par conséquent, elle recevra un contenu (une valeur).

Deux variables peuvent avoir la même valeur, mais une variable ne peut pas avoir plusieurs valeurs en même temps.

Par contre, la valeur d'une variable peut varier (sinon c'est une variable constante ou tout simplement constante) au cours du programme. Dans ce cas, l'ancienne valeur est tout simplement écrasée et remplacée par la nouvelle.

Un objet bien défini, est caractérisé par son nom (identificateur), sa valeur et son type.

1.1 Identificateur

C'est un nom qui est attribué à une variable et est représenté par une suite de caractères alphanumériques (sans espace) à condition que le premier caractère soit une lettre. Le nombre maximum de caractères autorisés pour un identificateur dépend du langage utilisé.



FIGURE 1 – Identificatuer d'une variable

Dans cette figure, vous avez beaucoup de conteneurs ; chacun a un identifiant qui permet son identification. Dans un programme, toutes les variables doivent avoir un identifiant permettant leur identification.

1.2 Type

L'ordinateur ne stockera pas de la même manière des objets différents de par leur nature. Ainsi, selon la nature de l'information contenue dans l'objet, l'ordinateur va coder d'une certaine façon. Cela veut dire que toutes les variables n'ont pas besoin de la même place en mémoire.

Par exemple, les lettres seront codées différemment aux nombres réels. Cette classification dans le codage spécifie l'ensemble des valeurs (constantes) que la variable peut prendre.

Le type d'une variable est défini par l'ensemble des constantes qu'elle peut avoir pour valeur et par l'ensemble des opérations auxquelles elle peut être soumise.

Le type d'un objet est limitatif : une variable ne peut prendre qu'un nombre limité de valeurs différentes. "Les limites dépendent aussi des langages".

Par exemple, des nombres trop grands ou des nombres irrationnels ne pourront pas être représentés.

1.3 Déclaration des variables

Pour qu'un programme puisse utiliser une variable, il faut au préalable que cette variable ait été déclarée, c'est-à-dire que le programme lui ait réservé une place en mémoire et ait attribué l'identificateur à cette place. Selon le type de l'objet, il faudra lui réserver plus ou moins de place : c'est pourquoi il faut déclarer le type des variables et pas seulement leur nom. Par ailleurs, selon le type des variables, les opérations possibles seront différentes. Donc la déclaration d'une variable indique deux choses :

- ⇒ son identificateur (son nom)
- ⇒ son type (sa taille, sa nature)

L'identificateur des variables doit être suffisamment signifiant pour qu'on reconnaisse leur fonction aisément. Par exemple pour des variables représentant un prix et une quantité, évitez a et b mais utilisez plutôt prix et quant.

1.4 Les types primitifs

En algorithmique, on distingue 4 types principaux :

- Le type **caractères** (lettres, chiffres, ponctuation, code des opérations, espace, retour chariot, ... et plus généralement toutes les touches que l'on peut trouver sur une machine à écrire)
- Le type **entier** (les nombres sans virgule)
- Le type **réel** (les nombres à virgule, dans certains langages, les entiers peuvent être considérés comme des réels mais avant le traitement, une conversion sera faite. Exemple : 4 qui est un entier sera converti en 4.0)
- Le type **booléen** (qui n'ont que deux valeurs possibles : soit VRAI, soit FAUX)

Les opérations (définies par un symbole qui dépend du langage) possibles sur les variables dépendent de leur type

Type	Opérations		
Entier	Symbole	Opération	Exemples
	+	Addition	2 + 23 vaut 25
	-	Soustraction	36 - 42 vaut -6
	*	Multiplication	4 * (-7) vaut -28
	DIV	Division entière	11 DIV 4 vaut 2
	MOD	Reste de la division entière	11 MOD 4 vaut 3
	^	Puissance	4 ^ 2 vaut 16

FIGURE 2 – Opérations du type Entier

Pour les **entiers**, la division est notée **div**. Elle est nommée division entière et diffère un peu de la division que l'on trouve sur les calculettes. Elle ne donne que le chiffre avant la virgule du résultat (elle renvoie un entier).

Les entiers supportent une opération supplémentaire appelée modulo, notée **mod** et qui renvoie le reste de la division entière.

Exemple :

- $7 / 2$ donne 3.5
- $7 \text{ div } 2$ donne 3
- $7 \text{ mod } 2$ donne 1

Type	Opérations		
Réel	Symbole	Opération	Exemples
	+	Addition	$2.8 + 23$ vaut 25.8
	-	Soustraction	$36.73 - 42.3$ vaut -5.57
	*	Multiplication	$4 * (-7)$ vaut -28.0
	/	Division réelle	$11 / 4$ vaut 2.75

FIGURE 3 – Operations du type Reel

Type	Opérations		
Booléen	Symbole	Opération	Exemples
	ET	Et logique	Voir table de vérité
	OU	Ou logique	Voir table de vérité
	NON	La négation	Voir table de vérité
	OUE	Le ou logique exclusif	Voir table de vérité

FIGURE 4 – Operations du type Booleen

ET logique :

Cet opérateur logique est binaire (il nécessite deux valeurs booléennes). On l'appelle la conjonction et il revoie la valeur booléenne VRAI si les deux valeurs booléennes sont VRAIES. Ci-après, on a la table de vérité.

ET	VRAI	FAUX
VRAI	VRAI	FAUX
FAUX	FAUX	FAUX

FIGURE 5 – ET logique

OU logique :

Cet opérateur logique est binaire (il nécessite deux valeurs booléennes). On l'appelle la disjonction et il revoie la valeur booléenne VRAI si au moins une des deux valeurs booléennes est VRAIE.

Ci-après, on a la table de vérité.

OU	VRAI	FAUX
VRAI	VRAI	VRAI
FAUX	VRAI	FAUX

FIGURE 6 – OU logique

NON logique :

Cet opérateur logique est unaire (il nécessite une valeur booléenne). On l'appelle la négation et il revoie le complément de l'opérande. Ci-après, on a la table de vérité.

NON	VRAI	FAUX
	FAUX	VRAI

FIGURE 7 – NON logique

OUE logique :

Cet opérateur logique est binaire (il nécessite deux valeurs booléenne). On l'appelle le ou exclusif; il peut être obtenu par une combinaison des opérateur **ET** et **OU** ; il revoie la valeur booléenne VRAI si les deux valeurs booléennes ont des états différents. Ci-après, on a la table de vérité.

OUE	VRAI	FAUX
VRAI	FAUX	VRAI
FAUX	VRAI	FAUX

FIGURE 8 – OUE logique

Les caractères sont comparés (*via les opérateurs de comparaison appelés aussi opérateur logique ou booléen sont détaillés dans le chapitre suivant*) selon l'ordre du code *ASCII* (American Standard Code for Information Interchange). C'est ainsi qu'on peut comparer tous les caractères entre eux. Par exemple la lettre **Z** (majuscule), de code ASCII 90 est inférieure à la lettre **a** (minuscule) de code ASCII 97. L'ordre ASCII des lettres de la même casse suit l'ordre alphabétique, de sorte que $A < B < C < D < \dots$

L'opérateur **&** sert à concaténer des chaînes de caractère, ce qui signifie transformer plusieurs

chaînes en une seule en les ajoutant les unes à la suite des autres.

– Ex : "Bonjour" & "à tous" donne "Bonjour à tous"

1.5 Syntaxe générale de l'algorithmique

Un algorithme est composé d'un en-tête (partie déclarative) et d'un corps où les différentes étapes de traitement sont effectuées.

ALGORITHME NomAlgo

CONSTANTES nombre = 10

► les constantes : il est obligatoire de leur donner une valeur dès leur déclaration

VARIABLES x1, x2 : réels

► les variables au sens strict

DEBUT

Instruction 1

Instruction 2

...

...

...

Instruction n

FIN

Les mots du langage algorithmique sont écrits en gras ou soulignés. Un algorithme commence par le mot **ALGORITHME** suivi de son identificateur (le nom du programme). Ensuite viennent les déclarations : dans un premier temps celles des constantes, annoncées par **CONSTANTES**, puis celle des variables, annoncée par **VARIABLES**. Pour déclarer une variable, on indique son identificateur suivi d'un double point et de son type. La valeur des constantes est donnée dès leur déclaration, avec le signe ← précédé de son identificateur.

Le corps du programme commence par **DEBUT** et se termine par **FIN**. On peut insérer des commentaires après le symbole ► jusqu'à la fin de la ligne.

2 INSTRUCTIONS ELEMENTAIRES

2.1 Généralités

L'exécution d'un programme est constituée :

- d'échanges d'informations en mémoire
- de calculs

Une instruction est un ordre élémentaire que peut exécuter directement l'ordinateur. Une instruction revient à déplacer une information d'un endroit à un autre de la mémoire. Les informations (données) manipulées par les instructions, peuvent prendre plusieurs formes :

- des variables
- des constantes
- des valeurs littérales (écrites telles qu'elles dans le programme : exemple "bonjour", 45, VRAI)
- des messages (libellés) envoyés à l'utilisateur (quelles données sont à entrer, quels résultats sont affichés), qui sont des valeurs littérales particulières
- des expressions complexes (combinaisons de variables, constantes et valeurs littérales avec des opérateurs) ex : $2 * r * 3.14$

Les instructions élémentaires les plus courantes sont :

- l'**affectation** : le fait de donner une nouvelle valeur à une variable
- l'**affichage** sur l'écran
- la **lecture** à travers le clavier

D'autres instructions permettent de lire et d'écrire sur d'autres périphériques : nous les étudierons plus tard.

2.2 Affectation

2.2.1 Présentation

L'affectation consiste tout simplement à placer une valeur dans une variable (ce qui revient à changer le contenu de cette variable). La nouvelle valeur est évaluée à partir d'une expression, qui peut être :

- soit une autre variable ou constante,
- soit une valeur littérale
- soit une combinaison de variables, de valeurs littérales et d'opérateurs

En algorithmique, pour représenter l'opération d'affectation, on va utiliser le symbole $\leftarrow$.

On évite d'utiliser le symbole $=$ pour ne pas le confondre avec l'égalité mathématique.

Ainsi l'instruction "affecter" à x la valeur signifie ranger dans x la valeur 15 et on écrit :

$x \leftarrow 15$

On peut avoir des exemples d'affectation du genre :

$x \leftarrow y$

$x \leftarrow 5 + 8$

$x \leftarrow 5 + y$

Les instructions d'affectation sont formées de deux parties :

A gauche du symbole $\leftarrow$, on trouve toujours le nom d'une variable destinée à recevoir une valeur et à droite, on trouve l'expression qui va donner la valeur qu'on veut affecter à la variable en question.

Tout d'abord elle détermine la valeur de l'expression à droite du $\leftarrow$ (la plupart du temps, il n'y a même pas de calculs à faire!) puis elle range le résultat dans la variable située à gauche.

2.2.2 Distinguer l'affectation de l'égalité mathématique

Quand on dit qu'une variable prend pour valeur une autre variable, ça ne veut pas dire qu'elles seront toujours égales! Cela veut seulement dire qu'au moment de l'instruction, la première variable va prendre la valeur de la seconde.

Voici les principales différences entre l'affectation et l'égalité mathématique :

L'instruction $x \leftarrow y$ n'a pas la même signification que l'instruction $y \leftarrow x$ contrairement aux mathématiques où $x = y$ est équivalent à $y = x$. L'instruction $x \leftarrow y$ signifie que x va prendre la valeur de y . Au contraire $y \leftarrow x$ signifie que y va prendre la valeur de x .

On ne peut affecter de valeur qu'à une variable. Il est impossible d'affecter une valeur à une expression. Ainsi, par exemple, l'instruction :

$a + 5 \leftarrow b$ n'a pas de sens,

alors que

$a + 5 = b$ a une signification en mathématiques.

Il est possible de retrouver la même variable à droite et à gauche de l'expression.

Ecrire : $a \leftarrow a + 3$ a un sens en programmation, alors que $a = a + 3$ n'a pas de sens en mathématiques. Cela signifie que l'on évalue $a + 3$ avec l'ancienne valeur de a et qu'on range le

résultat dans a. La valeur de a sera donc augmentée de 3 par cette opération.

Illustration :

ALGORITHME Exemple

VARIABLES x1, x2 : entiers

► x1 et x2 sont des entiers qui ont chacun une valeur indéterminée

DEBUT

x1 $\leftarrow$ 5 ► x1 vaut 5 et x2 a toujours une valeur indéterminée

x2 $\leftarrow$ x1 + 6 ► x2 vaut 11

x1 $\leftarrow$ x1 * x2 ► x1 devient 5 * 11. L'ancienne valeur de x1
est perdue

FIN

Application : la permutation

Comment échanger les valeurs de deux variables a et b ?

Élément de réponse : si on commence par écrire $a \leftarrow b$, on va détruire la valeur de a. (Il va falloir utiliser une troisième variable que l'on utilisera pour sauvegarder la valeur de a)

2.2.3 Compléments

L'affectation de caractères et de chaînes Les valeurs numériques, nous l'avons vu, s'écrivent de façon naturelle, sans avoir à les accompagner de symboles supplémentaires. Il n'en est pas de même avec les caractères et les chaînes, qu'il faut entourer de guillemets ou d'apostrophe. Sinon, en effet, le programme pourrait confondre le caractère ou la chaîne avec le nom d'une variable. Les chaînes sont entourées de guillemets et les caractères de simples apostrophes.

Exemple :

reponse $\leftarrow$ 'o'

mot $\leftarrow$ "bonjour"

L'initialisation des variables Au début d'un programme, les variables n'ont pas encore reçues de valeur ; on dit qu'elles sont indéfinies. Toutes les variables doivent être initialisées, c'est-à-dire recevoir une valeur initiale, avant leur utilisation (à droite d'une affectation ou dans une expression conditionnelle). Sinon, le contenu de la variable n'est pas vide, il n'est pas forcément nul, il est

quelconque. Ainsi, lorsqu'on utilise une variable indéfinie (qui n'a pas encore reçue de valeur), le comportement du programme va être aléatoire.

De là vient la nécessité de toujours bien initialiser les variables avant d'utiliser leur valeur. L'initialisation s'effectue généralement au début du programme, juste après les déclarations. Elle prend la forme d'une simple affectation ou d'une saisie au clavier.

Exemple :

ALGORITHME Initialisation

VARIABLES x1 : entiers

mot : chaîne

DEBUT

► Initialisation

x1 $\leftarrow$ 5 ► x1 vaut 5

mot $\leftarrow$ "bonjour" ► mot vaut bonjour

► Autres instructions

FIN

2.3 Entrée et Sortie



FIGURE 9 – Les périphériques

Parmi les composants d'un ordinateur, on distingue les périphériques de sortie (écran, imprimante, haut-parleurs, graveur de CD, ...), des périphériques d'entrée (clavier, souris, lecteur de CD/DVD Rom, scanner, micro, webcam, manette de jeu, appareil photo et caméscope numériques, /...). Il existe des périphériques particuliers qui se comportent comme des entrées et des sorties et en plus, on peut sauvegarder des informations : on les appelle des périphériques d'archivage.

Chaque composant joue un rôle bien précis : le microprocesseur reçoit et transmet des informations ; le disque dur stocke les informations ; les périphériques d'entrée analysent les informations fournies et les transmettent au microprocesseur ; les périphériques de sortie (imprimante, haut-parleurs, graveur) traduisent les informations reçues par les périphériques d'entrée, puis transmises au microprocesseur.

Alors, les entrées et les sorties permettent la communication avec le programme : ce sont les instructions d'écriture et de lecture.

2.3.1 L'instruction d'écriture (entrées)

L'instruction d'écriture (Lire) permet de communiquer des données au programme. Cette instruction assigne une valeur entrée au clavier dans une variable. Tant que l'utilisateur n'entre rien au clavier, le déroulement du programme est stoppé.

Remarque : notez que la saisie d'une variable au clavier permet, comme l'initialisation d'affecter une valeur initiale à une variable : donc les variables saisies n'ont pas à être initialisées.

Syntaxe : *Lire variable1 [, variables2,]*

Exemples :

Lire x

Cette instruction va lire la valeur saisie au clavier et l'affecte à la variable x

Lire x, y

Cette instruction lit la première valeur saisie au clavier et l'affecte à x, puis lit la deuxième valeur saisie et l'affecte à y.

Utilité de la saisie On pourrait se dire que l'instruction de saisie est inutile car on dispose déjà un moyen d'attribuer une valeur aux variables, par l'instruction d'affectation. Mais en fait, l'instruction de saisie (ou de lecture sur un périphérique autre que le clavier) est indispensable pour permettre d'utiliser le même programme sur des données différentes sans avoir à changer les valeurs du programme à chaque fois.

Par exemple, l'instruction lire x à laquelle on fournirait 5, pourrait être remplacée par $x \leftarrow 5$. Mais alors si on veut utiliser le programme avec une autre valeur, il faudra aller changer le programme. En revanche, si on utilise une instruction de saisie, le choix de la valeur se fait en cours d'exécution du programme. On peut donc utiliser le programme autant de fois que l'on veut avec des données différentes sans avoir à modifier le programme.

Exemple :

Le traitement qui consiste à calculer le périmètre d'un cercle à partir de son rayon peut s'effectuer pour toutes les valeurs possibles de rayon.

A chaque exécution du programme, on fournit une valeur différente au rayon, ce qui va donner un périmètre différent.

2.3.2 L'instruction de lecture (sortie)

La plupart des programmes nécessitent de communiquer à l'utilisateur un certain nombre de résultats par l'intermédiaire d'un périphérique. Pour cela, ils utilisent des instructions d'affichage. L'instruction d'affichage (sortie) permet de fournir des résultats sous forme directement compréhensible pour l'utilisateur à travers l'écran.

Syntaxe : *Afficher expression1, [expression2]*

Exemples :

Afficher technicien

Cette instruction permet d'afficher la valeur de la variable technicien à l'écran. Si technicien est une chaîne qui vaut " moussa", cette instruction affichera moussa à l'écran.

Afficher "Bonjour"

Celle-ci permet d'afficher la chaîne littérale Bonjour à l'écran

Afficher a, b

Quand on veut afficher deux objets à la suite, on les sépare d'une virgule. Si a vaut 12 et b vaut 7, on obtient alors à l'écran : 12 7

Remarque : dans la plupart des langages (dont le C++), il faut ajouter dans le programme un espace entre les différentes expressions à afficher, mais en algorithmique, on ne s'embête avec ces détails.

On peut mélanger l'affichage de valeurs littérales et de variables. Cela est particulièrement utile si on veut voir apparaître un libellé (texte accompagnant la saisie des données ou l'édition des résultats, permettant de guider l'utilisateur).

Exemple :

Afficher "Voici les résultats : x = ", x, " et y = ", y Ils apparaîtront dans l'ordre. Nous aurons donc à l'écran (en supposant que les valeurs de x et y sont respectivement 5 et 10) : Voici les résultats : x = 5 et y = 10

2.3.3 Synthèse

Réaliser un algorithme, c'est exprimer, dans un langage précis (le langage algorithmique), les règles de traitement d'un problème pour le soumettre à un ordinateur (par un programme).

Les données d'un programme sont mémorisées dans des variables qui sont des cases mémoire. Une variable est un contenant, qui contient une et une seule valeur à un moment donné.

Une variable est caractérisée par :

- un identificateur (son nom),
- une valeur (qui peut varier au cours du programme, sauf pour les constantes)
- un type (qui détermine sa taille et les opérations possibles)

Les instructions de base sont :

- l'affectation permet de changer la valeur d'une variable.
- la saisie permet d'assigner à une variable, une valeur entrée au clavier
- l'affichage permet d'écrire le contenu d'une variable ou d'une expression à l'écran

Exemple complet : Nous allons écrire un algorithme qui calcule l'âge de l'utilisateur en fonction de son année de naissance.

ALGORITHME age

CONSTANTES annee $\leftarrow$ 2021

VARIABLES ddn : entiers $\blacktriangleright$ date de naissance

DEBUT

Afficher "Entrez votre année de naissance sur 4 chiffres"

Lire ddn

Afficher "Vous avez, ou aurez dans l'année" , année - ddn , "ans."

FIN